

Security Architecture Report

Objective

This report restructures the attached source-code trace into a concise security report covering the end-to-end authentication and authorization flow, the six-layer server-side enforcement chain, OpenFGA and Vault integration, API authorization coverage, and the main security design principles of the libcloud REST platform.[file:6]

Introduction

The attached material describes a security architecture split across two repositories: `openfga_my/` for bootstrap scripts and infrastructure orchestration, and `libcloud.rest/` for the REST API that enforces authorization on every request.[file:6] The overall design combines Dex-based OIDC authentication, JWT-derived identity resolution, scope and provider gates, OpenFGA fine-grained authorization, Vault-backed credential retrieval, and cloud-provider execution through libcloud drivers.[file:6]

Emphasis: The most important architectural fact is that the client only performs authentication and request assembly. The actual security boundary lives on the server, where every cloud operation passes through multiple independent gates before any backend credential is fetched or any provider API is called.[file:6]

Architecture Overview

The security model is layered across the client, OpenFGA, Vault, and the REST API service. [file:6] The client script `provision_aws.sh` performs Step 1 only, while Steps 2 through 5 run inside the server for every request, which prevents the client from directly handling backend cloud credentials or making authoritative authorization decisions.[file:6]

Step	Function	Enforcement point	Repository
1	Authenticate with Dex and obtain OIDC token.[file:6]	Client script to Dex IdP.[file:6]	<code>openfga_my/</code> .[file:6]
2	Extract identity and resolve principal plus group semantics.[file:6]	<code>oidc_service.py</code> . [file:6]	<code>libcloud.rest/</code> . [file:6]
3	Run OpenFGA checks before each cloud operation. [file:6]	<code>policy.py</code> .[file:6]	<code>libcloud.rest/</code> . [file:6]
4	Fetch server-side backend credentials.[file:6]	<code>credentials.py</code> . [file:6]	<code>libcloud.rest/</code> . [file:6]
5	Execute cloud operation through libcloud/provider APIs.[file:6]	Service layer.[file:6]	<code>libcloud.rest/</code> . [file:6]

The architecture also includes `openfga_bootstrap.py`, which seeds the OpenFGA model and tuples, and `vault_bootstrap.py`, which seeds secrets into Vault KV v2 for later server-side

retrieval.[file:6]

Client-Side Flow

The client-side path is intentionally narrow. `provision_aws.sh` calls `idp_login`, which is implemented in `common.sh` and obtains an access token by invoking `idp_login.py` to drive an OIDC Authorization Code flow against Dex.[file:6]

Within `idp_login.py`, the flow includes `dex_login()` for the Dex authorization page and callback capture, `_exchange_code()` for token exchange at `/dex/token`, `_save_cache()` for token caching, and `_refresh()` for token refresh on later runs.[file:6] The script also maps script-level usernames such as `reader`, `provisioner`, and `outsider` to underlying LLDAP UUIDs through the `USER_UUID` table.[file:6]

Client function	Purpose
<code>idp_login()</code>	Starts Dex login and exports the access token for API calls.[file:6]
<code>dex_login()</code>	Performs browser-style auth flow and captures authorization code.[file:6]
<code>_exchange_code()</code>	Exchanges code for access and refresh tokens at Dex token endpoint.[file:6]
<code>_save_cache()</code>	Writes token cache to <code>generated/tokens/<username>.json</code> . [file:6]
<code>_refresh()</code>	Refreshes cached token instead of re-running full login flow.[file:6]
<code>libcloud_api()</code>	Attaches Bearer token and optional <code>X-Provider-Connection</code> header to each REST call. [file:6]

A notable client design choice is that the `X-Provider-Connection` header carries only routing context such as `provider`, `region`, and `auth_binding`, and not any backend secret material.[file:6] A client-side OpenFGA preview helper exists in `common.sh`, but it is commented out and explicitly described as a convenience check rather than a real security gate.[file:6]

Emphasis: The client can suggest a target backend through `auth_binding`, but it cannot supply backend secrets or bypass authorization. This sharply limits the damage a compromised client script can do, because the authoritative checks and secret resolution remain server-side.[file:6]

Server Enforcement Chain

Every cloud request passes through a six-layer authorization chain inside `libcloud.rest`, and any layer can stop the request before it reaches a cloud provider.[file:6]

Layer Summary

Layer	Purpose	Main file
0	Extract Bearer token from the HTTP Authorization header.[file:6]	<code>dependencies.py</code> . [file:6]
1	Decode JWT, validate signature, resolve stable principal, assign scopes and allowed providers.[file:6]	<code>oidc_service.py</code> , <code>identity.py</code> . [file:6]
2	Enforce token scope and provider allowlist.[file:6]	<code>policy.py</code> . [file:6]

Layer	Purpose	Main file
3	Reject client-supplied backend credentials unless explicitly allowed. [file:6]	credentials.py.[file:6]
4	Enforce OpenFGA checks such as can_connect, can_use, can_read, and can_provision.[file:6]	policy.py, fga_client.py. [file:6]
5	Resolve backend credentials from Vault or environment fallback.[file:6]	credentials.py, vault_client.py.[file:6]
6	Execute libcloud driver call against AWS or Nutanix APIs.[file:6]	Service and route layers.[file:6]

Layer 0 uses FastAPI bearer extraction and rejects missing or malformed bearer tokens with `auth_invalid_token`.
[file:6] Layer 1 validates the JWT against Dex JWKS, resolves the stable principal through principal maps and legacy aliases, assigns scopes and provider allowlists, and preserves the raw bearer token so it can be forwarded to OpenFGA later.
[file:6]

Layer 2 enforces the requested endpoint scope and provider allowlist before OpenFGA is consulted, which means a user blocked at the scope or provider stage never reaches backend authorization checks.
[file:6] Layer 3 prevents the client from submitting cloud credentials directly unless `allow_client_credentials` is enabled, forcing the platform to use server-side identities via Vault or controlled environment variables.
[file:6]

Layer 4 runs OpenFGA checks on every request, typically validating `can_connect` on `libcloud_api:main`, `can_use` on a provider object such as `provider:aws`, and either `can_read` or `can_provision` on a backend object such as `aws_region:aws`.
[file:6] Layer 5 then resolves credentials based on `auth_binding`, preferring Vault KV v2 and falling back to environment variables only for development use.
[file:6]

OpenFGA Model and Authorization Logic

The attached trace documents a tenant-centric OpenFGA model with the active object types `user`, `platform`, `tenant`, `libcloud_api`, `provider`, `aws_region`, and `nutanix_cluster`.
[file:6] The model relies on a small number of direct role-bearing relations and then computes operational permissions such as `can_connect`, `can_use`, `can_read`, and `can_provision` through unions, intersections, and tuple-to-user set propagation.
[file:6]

Object type	Authorization role in the model
platform	Holds superadmin and <code>can_manage_platform</code> . [file:6]
tenant	Holds owner, admin, viewer, and derived relations including <code>can_provision</code> and <code>can_read</code> . [file:6]
libcloud_api	Governs top-level API access through <code>can_connect</code> . [file:6]
provider	Governs cloud-provider usage through <code>can_use</code> . [file:6]
aws_region	Represents AWS backend objects, with derived read and provision permissions. [file:6]
nutanix_cluster	Mirrors <code>aws_region</code> for Nutanix backends. [file:6]

A key security property is that `can_provision` on backend objects is an intersection, not a simple role union.[file:6] The caller must satisfy both a role condition such as `operator`, `tenant_admin`, or `tenant_owner`, and a provider condition through `provider#can_use`, which is what enforces cloud-specific isolation between AWS and Nutanix.[file:6]

The bootstrap seeds 17 initial tuples, including platform superadmin, tenant-level owner/admin/viewer assignments, and structural links tying tenants to the API, providers, and backends.[file:6] The model then scales by adding user-role assignments rather than per-user per-endpoint tuples, keeping the tuple count proportional to users and roles instead of API surface size.[file:6]

Emphasis: OpenFGA is not just a post-login role lookup; it is the runtime decision engine that determines whether a principal may connect to the API, use a provider, and read or provision a specific backend object. This turns tenant isolation and least privilege into graph relationships rather than route-local logic.[file:6]

API Authorization Coverage

The attached trace maps the full API surface to scopes and OpenFGA relations.[file:6] Compute, network, storage, connections, jobs, auth, and providers endpoints are all listed with their required scopes and whether they rely on `can_read`, `can_provision`, or auxiliary ownership checks.[file:6]

Endpoint Summary by Surface

API surface	Example endpoints	Main scope pattern	OpenFGA pattern
Compute	<code>/locations</code> , <code>/nodes</code> , <code>/images</code> , <code>/volumes</code> , <code>/snapshots</code> , <code>/key-pairs</code> . [file:6]	Read scopes for enumeration, write/manage scopes for mutation. [file:6]	<code>can_read</code> for reads, <code>can_provision</code> for writes.[file:6]
Network	<code>/networks</code> , <code>/subnets</code> , <code>/security-groups</code> , <code>/load-</code> <code>balancers</code> , <code>/floating-ips</code> . [file:6]	<code>compute:network:read</code> or <code>compute:network:manage</code> . [file:6]	<code>can_read</code> for reads, <code>can_provision</code> for writes.[file:6]
Storage	<code>/buckets</code> , object listing, upload, delete, download.[file:6]	Uses <code>compute:read</code> or <code>network-</code> <code>manage</code> scopes depending on route. [file:6]	<code>can_read</code> for reads, <code>can_provision</code> for writes.[file:6]
Connections	POST <code>/v1/connections:test</code> . [file:6]	<code>compute:read</code> . [file:6]	<code>can_read</code> . [file:6]
Jobs	GET <code>/v1/jobs/{job_id}</code> . [file:6]	<code>jobs:read</code> . [file:6]	Additional owner/admin check rather than generic backend relation alone. [file:6]
Auth	<code>/logout</code> , <code>/me</code> , <code>/token/introspect</code> . [file:6]	Bearer-only or admin scope depending on route. [file:6]	Not all auth routes map to backend OpenFGA checks. [file:6]

API surface	Example endpoints	Main scope pattern	OpenFGA pattern
Providers	GET /v1/providers.[file:6]	Public.[file:6]	No auth required.[file:6]

A consistent pattern appears across the API surface: read endpoints generally permit either explicit read scopes or broader aliases such as `compute:read`, while mutating endpoints require write or `:manage` scopes and `can_provision` on the backend object.[file:6]

Error Handling and Operational Controls

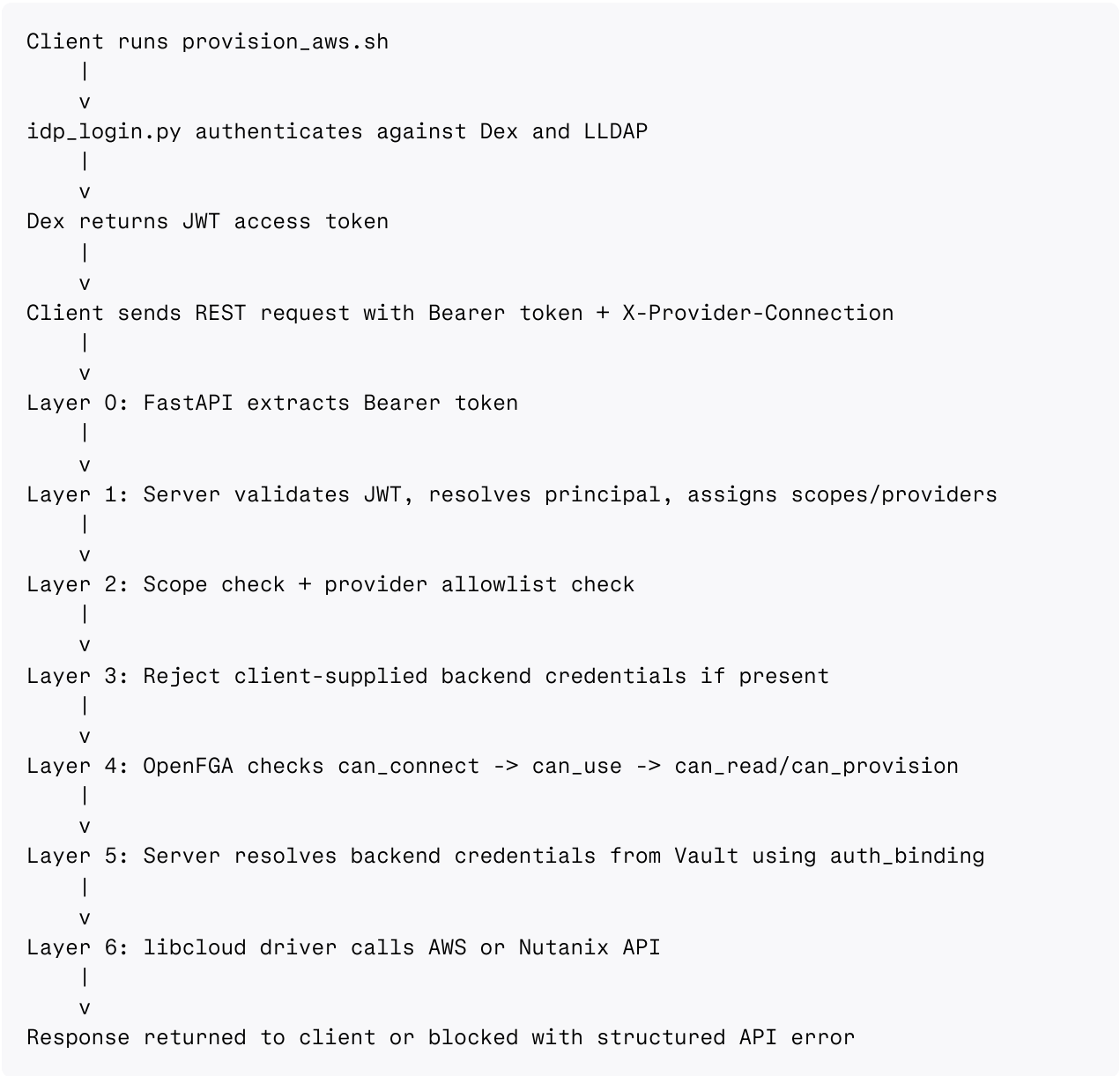
The system uses structured `APIError` responses with machine-readable codes such as `auth_invalid_token`, `auth_insufficient_scope`, `auth_provider_denied`, `authz_fga_denied`, `server_credentials_missing`, and `server_credentials_unavailable`.[file:6] These cover failures in token handling, identity resolution, scope gating, provider restrictions, OpenFGA decisions, job ownership checks, and Vault connectivity or secret lookup.[file:6]

HTTP status	Error category	Example code	Trigger
401	Authentication failure	<code>auth_invalid_token</code> .[file:6]	Missing, malformed, or invalid JWT.[file:6]
401	Token expiry	<code>auth_expired_token</code> .[file:6]	JWT is expired.[file:6]
403	Scope violation	<code>auth_insufficient_scope</code> .[file:6]	Token lacks route-required scope.[file:6]
403	Provider restriction	<code>auth_provider_denied</code> .[file:6]	Principal is not allowed to target that provider. [file:6]
403	Credential policy violation	<code>auth_client_credentials_forbidden</code> .[file:6]	Client attempts to pass backend credentials. [file:6]
403	Identity resolution failure	<code>auth_user_unknown</code> .[file:6]	OIDC principal cannot be mapped.[file:6]
403	OpenFGA denial	<code>authz_fga_denied</code> .[file:6]	OpenFGA returns <code>allowed: false</code> . [file:6]
503	OpenFGA service issue	<code>authz_fga_error</code> , <code>authz_fga_unavailable</code> . [file:6]	OpenFGA error or connectivity failure. [file:6]
503	Vault credential issue	<code>server_credentials_missing</code> , <code>server_credentials_unavailable</code> .[file:6]	Missing or unreachable backend secrets.[file:6]

These error classes reinforce a defense-in-depth design because each layer fails distinctly and observably, making troubleshooting and audit trails more precise.[file:6]

Flowchart

The following text flowchart summarizes the end-to-end authorization path for a mutating request such as aws-admin calling POST /v1/compute/nodes.[file:6]



Design Principles

The source-code trace highlights several explicit design principles built into the platform. [file:6] Client-supplied cloud credentials are rejected, OpenFGA checks are performed on every request, authorization is split into scope, provider, and backend-resource layers, tenant isolation is keyed by auth_binding, and OpenFGA receives the caller's Dex JWT so tuple-store access itself remains OIDC-protected.[file:6]

Principle	Meaning
Server-side secret ownership	Clients pass tenant selectors, not backend credentials.[file:6]
Per-request authorization	Every cloud operation triggers a fresh OpenFGA check.[file:6]

Principle	Meaning
Three-tier authorization	Scope gate, provider gate, and backend object authorization work together.[file:6]
Tenant isolation by binding	<code>auth_binding</code> maps to per-tenant backend objects and Vault secrets.[file:6]
OIDC-forwarded OpenFGA calls	OpenFGA validates the same Dex-issued JWT used by the API caller.[file:6]
Scalable tuple model	Structural tuples are seeded once; permissions propagate through computed relations.[file:6]
Provider extensibility	New clouds require object-type registry, model entry, and driver support rather than policy rewrites.[file:6]

Emphasis: The architecture is secure by composition rather than by any single control. Even if one layer is misconfigured, an attacker still has to pass separate JWT validation, scope checks, provider restrictions, OpenFGA graph authorization, and Vault-backed credential resolution before any cloud-side action can occur.[file:6]

Conclusion

The attached source-code trace describes a well-structured, layered security architecture in which authentication, authorization, secret management, and cloud execution are clearly separated across components and repositories.[file:6] The client is intentionally thin, while the REST API becomes the central enforcement point for JWT validation, identity resolution, provider restriction, OpenFGA authorization, and backend credential handling.[file:6]

Operationally, the strongest properties of this design are its defense in depth, tenant isolation through backend object bindings, rejection of client-supplied cloud secrets, and the use of OpenFGA as a runtime graph authorizer instead of relying only on route-local role checks.[file:6] The result is a platform that is easier to audit, safer to extend to new providers, and more resilient against both client compromise and authorization drift because permissions are derived from a small, explicit policy graph rather than scattered across endpoint implementations.[file:6]